Excel output

Study guide 2: core/excel.py and the app layouts in commission/excel.py and activity/excel.py

What this guide covers

File	Lines	Job
`core/excel.py`	68	How every report's Excel looks: SheetSpec, style_sheet, workbook_bytes, col_numbers
`commission/excel.py`	20	Which sheets and bands the commission report has
`activity/excel.py`	25	The same for client activity, with bands that follow optional columns

The idea in one line

> **Separation**
> An app's `excel.py` **describes** its sheets; `core/excel.py` **draws** them. Change the gold header in one place and every report changes.

How to use it

- Look at the layout picture in Part 2 first: rows, bands and the 0-based/1-based switch cause most Excel bugs.
- Rebuild core/excel.py in Part 3's order, then the two app layouts.
- Run the checkpoints in Part 5. They open the bytes with openpyxl and check real cells.

Part 1. Think before you write code

The job

core/excel.py
Turn a list of described sheets into one finished .xlsx file, in memory, with the MIS look: optional coloured bands, gold headers, borders, money formats, frozen headers and filter buttons.

The contract

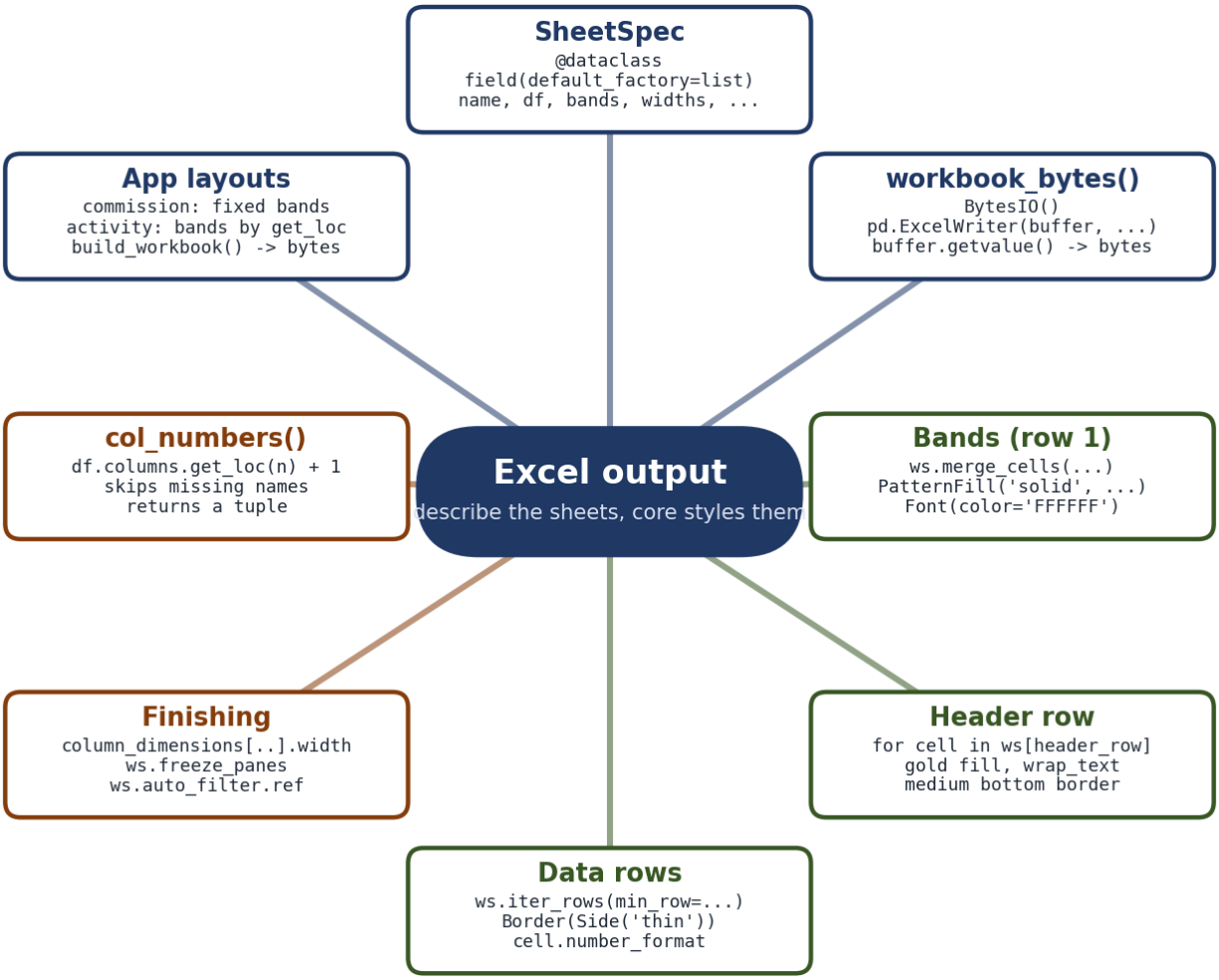
```
@dataclass
class SheetSpec:                      # what one sheet should look like
    name: str
    df: pd.DataFrame
    bands: list = ...                 # [(title, first col, last col, colour), ...]
    widths: list = ...
    money_cols: tuple = ()
    center_cols: tuple = ()

def workbook_bytes(sheets: list[SheetSpec]) -> bytes: ...     # the finished file
```

Facts that shape the design

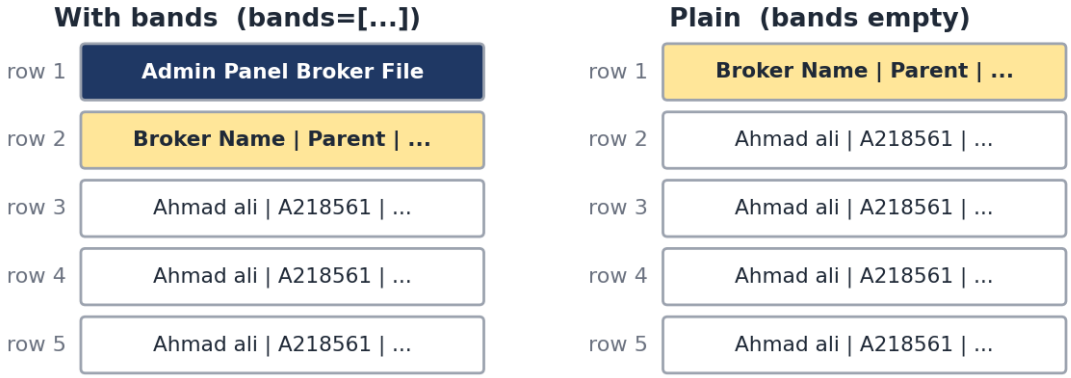
Fact	So the code...	Piece
Two people may run reports at once	never writes to a fixed path	BytesIO + return bytes
Each report has its own sheets and bands	takes a description, not hard-coded layouts	SheetSpec
Some sheets have a band row, some don't	works out the header row from the spec	header_row = 2 if spec.bands else 1
pandas counts rows from 0, Excel from 1	converts in exactly one place	startrow=1 if bands else 0
Optional columns move later columns	finds columns by name	col_numbers(df, names)
pandas writes plain cells only	styles the sheet pandas just wrote	writer.sheets[name] + openpyxl

The whole module as a mindmap



Part 2. The big picture

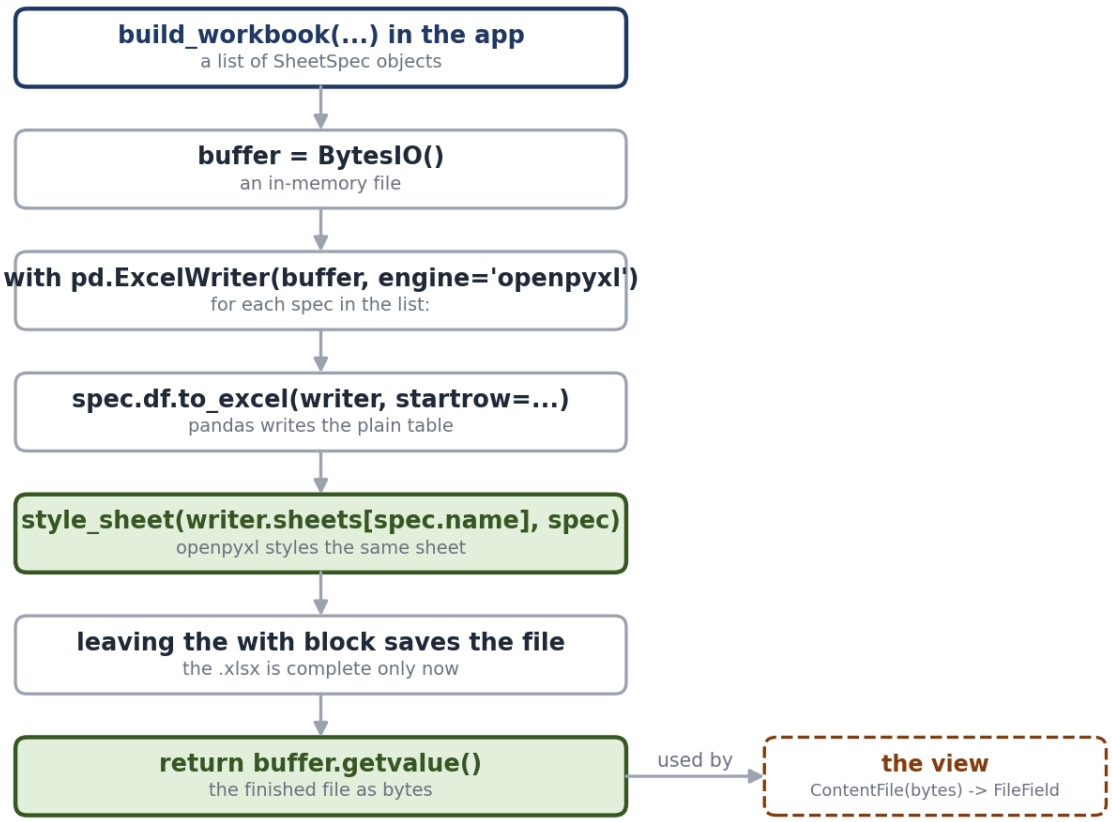
Rows, bands and the two counting systems




```
to_excel(startrow=1)  ->  header on Excel row 2          to_excel(startrow=0)  ->  header on Excel row 1
              freeze_panes = 'B3'                                       freeze_panes = 'B2'
```

pandas' `startrow` counts from 0, so `startrow=1` leaves the first Excel row empty for the band. openpyxl counts from 1, so the band is `row 1` and the header is `row 2`. Every row number in `style_sheet` is an Excel number.

From description to bytes



Part 3. Build it step by step

Step 1. SheetSpec: describe one sheet

What I was thinking
"Five or six things travel together for every sheet: its name, its table, its bands, widths, money and centred columns. Passing them as loose arguments would make every call long and easy to get wrong."
"A dataclass gives a named bundle with defaults, and Python writes the constructor for me."
"Lists as defaults need field(default_factory=list); a plain [] default is shared between objects, and dataclasses refuse it."

```
from dataclasses import dataclass, field

import pandas as pd


@dataclass
class SheetSpec:
    """One sheet of a workbook and how to style it."""
    name: str
    df: pd.DataFrame
    bands: list = field(default_factory=list)      # [(title, first col, last col, colour), ...]; empty = no band row
    widths: list = field(default_factory=list)     # column widths in order; missing ones get 14
    money_cols: tuple = ()                         # 1-based column numbers
    center_cols: tuple = ()
```

Tool	What it does	Why here
`@dataclass`	Generates __init__ from the annotated fields	SheetSpec('Brokers', df, bands=...) just works
`name: str`	A field with a type hint	Required: no default
`field(default_factory=list)`	A new empty list for every object	Each sheet gets its own bands list
`money_cols: tuple = ()`	An immutable default	Safe to share; can't be changed

Check yourself
SheetSpec('X', df) works with only two arguments, and SheetSpec('X', df).bands is an empty list.

Try the mistake once
Write `bands: list = []` and import the module: Python raises `ValueError: mutable default <class 'list'> for field bands is not allowed: use default_factory`. The dataclass protects you from the shared-default bug.

Step 2. workbook_bytes: write every sheet into memory

What I was thinking
"The view needs the file as bytes to store it. BytesIO behaves like a file but lives in memory."
"pandas writes each table; right after, writer.sheets[name] is the openpyxl sheet, so I can style it before the file is saved."
"The file is only finished when the with block ends, so getvalue() must come after it."

```
from io import BytesIO


def workbook_bytes(sheets: list[SheetSpec]) -> bytes:
    """Write every sheet with its style and return the finished .xlsx as bytes."""
    buffer = BytesIO()
    with pd.ExcelWriter(buffer, engine="openpyxl") as writer:
        for spec in sheets:
            spec.df.to_excel(writer, sheet_name=spec.name, index=False, startrow=1 if spec.bands else 0)
            style_sheet(writer.sheets[spec.name], spec)
    return buffer.getvalue()
```

Tool	What it does	Why here
`BytesIO()`	A file-like object in memory	No path, no clashes between users
`pd.ExcelWriter(buffer, engine='openpyxl')`	One workbook, several sheets	openpyxl, so the styling calls work
`with ... as writer:`	Saves and closes the workbook on exit	The zip file is written at this point
`to_excel(..., index=False, startrow=...)`	Writes the table, from a 0-based row	Leaves row 1 free when there are bands
`writer.sheets[spec.name]`	The openpyxl worksheet just written	Styled before the file is saved
`buffer.getvalue()`	Everything written so far, as bytes	Called after the with block: the file is complete

Check yourself
The result starts with b'PK': an .xlsx file is a zip archive. pd.read_excel(BytesIO(data), sheet_name='X') reads it back.

Step 3. style_sheet, part A: the header row and the bands

What I was thinking
"Where is the header? Row 2 if there are bands, otherwise row 1. Work it out once and use it everywhere below."
"I expected a merged range to keep each cell's own look, so I filled every cell in it, then put the title and font on the first one." A test later showed that half of this wasn't needed: see the note after this step.

```
from openpyxl.styles import Alignment, Border, Font, PatternFill, Side


def style_sheet(ws, spec: SheetSpec):
    thin = Side(style="thin", color="D0D5DD")
    header_row = 2 if spec.bands else 1
    for title, c1, c2, colour in spec.bands:
        ws.merge_cells(start_row=1, start_column=c1, end_row=1, end_column=c2)
        for c in range(c1, c2 + 1):
            ws.cell(1, c).fill = PatternFill("solid", fgColor=colour)
        ws.cell(1, c1).value = title
        ws.cell(1, c1).font = Font(name="Arial", bold=True, size=12, color="FFFFFF")
        ws.cell(1, c1).alignment = Alignment(horizontal="center", vertical="center")
        ws.row_dimensions[1].height = 24
```

Tool	What it does	Why here
`header_row = 2 if spec.bands else 1`	Picks the header row once	An empty list counts as False
`for title, c1, c2, colour in spec.bands`	Unpacks each 4-item tuple	Readable names instead of b[0], b[1]
`ws.merge_cells(start_row=..., ...)`	Joins a range into one cell	One title over several columns
`range(c1, c2 + 1)`	c1 to c2 inclusive	Fills every cell (turned out unnecessary: see below)
`ws.cell(row, column)`	A cell by numbers, 1-based	No letter arithmetic
`PatternFill('solid', fgColor=...)`	A solid background colour	Hex without the #
`Font(...), Alignment(...)`	Text style and position	White bold, centred titles
`ws.row_dimensions[1].height`	The height of row 1	Room for the band title

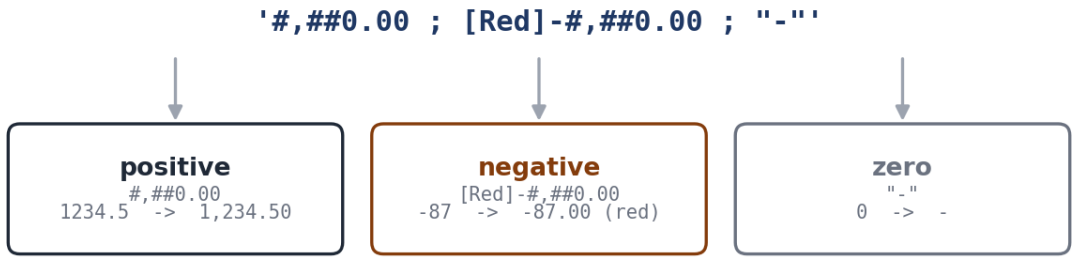
Check yourself
With bands [('Band', 1, 2, '1F3864')], 'A1:B1' is in ws.merged_cells and A1 has the navy fill.

What a test taught me here
The checkpoint first asserted that B1 kept its navy fill. It failed: after saving, B1 reads back as a `MergedCell` with no fill. openpyxl turns every cell of a merged range except the first into a placeholder, and Excel paints the whole merged area with the **first** cell's value and style.
A workbook with only A1 filled, opened in LibreOffice, showed a solid band across all four columns. So the inner loop is harmless but does nothing visible. Exercise: replace it with one line, `ws.cell(1, c1).fill = PatternFill('solid', fgColor=colour)`, and rerun the checkpoints.
The habit: when you believe something about a library, write the test that would prove it.

Step 4. style_sheet, part B: header cells and data cells

What I was thinking
"ws[row_number] gives every cell in that row: style the headers in one loop."
"For data, iter_rows walks every row below the header. Each cell gets a border; money columns get the format; some columns are centred."
"Column numbers in money_cols are 1-based Excel numbers, the same as cell.column."

```
    for cell in ws[header_row]:
        cell.font = Font(name="Arial", bold=True, size=10)
        cell.fill = PatternFill("solid", fgColor="FFE699")
        cell.alignment = Alignment(horizontal="center", vertical="center", wrap_text=True)
        cell.border = Border(left=thin, right=thin, top=thin, bottom=Side(style="medium"))
    for row in ws.iter_rows(min_row=header_row + 1, max_row=ws.max_row):
        for cell in row:
            cell.font = Font(name="Arial", size=10)
            cell.border = Border(left=thin, right=thin, top=thin, bottom=thin)
            if cell.column in spec.center_cols:
                cell.alignment = Alignment(horizontal="center")
            if cell.column in spec.money_cols:
                cell.number_format = MONEY_FORMAT
```



Sections are separated by ; and apply to positive, negative and zero values in that order.

Tool	What it does	Why here
`ws[header_row]`	A tuple of the row's cells	Loop over the headers
`wrap_text=True`	Long headers break onto two lines	'Total Negative Balance Protection' fits
`Side(style='thin', color=...)`	One edge of a border	Made once, reused for every cell
`Border(left=, right=, top=, bottom=)`	All four edges	A medium bottom edge under the headers
`ws.iter_rows(min_row=, max_row=)`	Rows of cells in a range	Every data row, skipping the header
`ws.max_row`	The last row with data	Where the loop stops
`cell.column`	The cell's column number (1-based)	Compared with money_cols
`cell.number_format = MONEY_FORMAT`	How Excel displays the value	The value stays a real number

Check yourself
The cell under 'Brokerage' on the first data row has number_format equal to MONEY_FORMAT.

Why a number format, not rounding in pandas
Rounding in pandas changes the value; `number_format` only changes the display. Excel's SUM still adds the exact values, and text cells like 'NO BROKERAGE' simply ignore the format.

Step 5. style_sheet, part C: widths, frozen headers, filters

What I was thinking
"Widths are set per column letter. Let openpyxl turn a number into a letter instead of building 'A', 'B'... myself."
"A missing width shouldn't crash: give it 14."
"Freeze below the header and right of column A, so the first column and the headers stay visible."

```
    n_cols = len(spec.df.columns)
    for i in range(1, n_cols + 1):
        width = spec.widths[i - 1] if i <= len(spec.widths) else 14
        ws.column_dimensions[ws.cell(header_row, i).column_letter].width = width
    ws.row_dimensions[header_row].height = 36
    ws.freeze_panes = ws.cell(header_row + 1, 2).coordinate
    ws.auto_filter.ref = f"A{header_row}:{ws.cell(header_row, n_cols).column_letter}{max(ws.max_row, header_row)}"
```

Tool	What it does	Why here
`spec.widths[i - 1] if ... else 14`	The width for column i, or a default	widths is 0-based, columns 1-based
`cell.column_letter`	'A', 'B', ..., 'AA'	No manual letter arithmetic
`ws.column_dimensions[letter].width`	Sets one column's width	In Excel's character units
`cell.coordinate`	'B3' for row 3, column 2	The top-left cell that is not frozen
`ws.freeze_panes = 'B3'`	Freezes rows above 3 and columns left of B	Headers and names stay on screen
`ws.auto_filter.ref = 'A2:K40'`	Filter buttons over that range	Starts at the header row
`max(ws.max_row, header_row)`	At least the header row	An empty table still gets a valid range

Check yourself
A banded sheet has freeze_panes 'B3'; a plain one 'B2'.

Step 6. col_numbers: find columns by name

What I was thinking
"Activity's columns shift when Name or Manager is missing, so hard-coded numbers would format the wrong column."
"get_loc gives the 0-based position; Excel needs 1-based, so add 1. Skip names that aren't there instead of failing."

```
def col_numbers(df: pd.DataFrame, names) -> tuple:
    """1-based column numbers of the named columns that exist, for money_cols / center_cols."""
    return tuple(df.columns.get_loc(n) + 1 for n in names if n in df.columns)
```

Tool	What it does	Why here
`df.columns.get_loc(name)`	A column's 0-based position	Where the column is now
`+ 1`	0-based to 1-based	Matches cell.column
`if n in df.columns`	Skips missing names	Optional columns never raise KeyError
`tuple(... for ...)`	A generator collected into a tuple	Matches the SheetSpec field type

Check yourself
col_numbers(pd.DataFrame(columns=['A', 'Deposit']), ['Deposit', 'Missing']) is (2,).

Step 7. The app layouts: fixed and dynamic

What I was thinking
"Commission's columns never change, so its bands and number columns can be constants at the top of the file."
"Activity's information columns vary (Client ID always; Name, Manager, Assigned To only if uploaded). Everything after them moves. 'Bills' is the first calculated column, so its position tells me how many information columns there are."
"The Manager Summary sheet only exists when there is a summary, so build the list, then append conditionally."

```
# commission/excel.py
BROKER_BANDS = [("Admin Panel Broker File", 1, 4, "1F3864"),
                ("Admin Panel Ledger File", 5, 8, "375623"),
                ("MIS Calculation", 9, 11, "843C0C")]

def build_workbook(brokers, clients) -> bytes:
    return workbook_bytes([
        SheetSpec("Brokers", brokers, bands=BROKER_BANDS, widths=[26, 12, 40, 11, 15, 15, 14, 17, 11, 16, 16],
                  money_cols=(5, 6, 7, 8, 10, 11), center_cols=(4, 9)),
        SheetSpec("Clients", clients, bands=CLIENT_BANDS, ...),
    ])

# activity/excel.py
def _client_bands(df):
    n_info = df.columns.get_loc("Bills")                    # Client ID + optional columns
    return [("Client List", 1, n_info, "1F3864"),
            ("Trading", n_info + 1, n_info + 2, "375623"),
            ("Money (month)", n_info + 3, n_info + 5, "375623"),
            ("Result", n_info + 6, n_info + 6, "843C0C")]

def build_workbook(clients, summary=None) -> bytes:
    sheets = [SheetSpec("Clients", clients, bands=_client_bands(clients),
                        widths=[14] + [22] * (clients.columns.get_loc("Bills") - 1) + [10, 12, 14, 14, 14, 13],
                        money_cols=col_numbers(clients, MONEY), ...)]
    if summary is not None:
        sheets.append(SheetSpec("Manager Summary", summary, ...))
    return workbook_bytes(sheets)
```

Tool	What it does	Why here
`BROKER_BANDS = [...]` (UPPER_CASE)	A module constant	Read by humans as 'don't change at runtime'
`get_loc('Bills')`	Number of columns before Bills	0-based position = count of columns before it
`n_info + 1, ...`	Band edges relative to Bills	Bands move with the columns
`[14] + [22] * (k) + [...]`	List arithmetic	One width per info column, then fixed widths
`if summary is not None`	Checks for 'no summary'	Not 'if summary': a DataFrame's truth is ambiguous

Check yourself
With only 'Client ID' and 'Manager' before Bills, _client_bands gives (1, 2), (3, 4), (5, 7), (8, 8).

Why 'is not None' and not 'if summary'
`if summary:` on a DataFrame raises `ValueError: The truth value of a DataFrame is ambiguous.` `is not None` asks the only question that matters here: was a summary made?

Part 4. Mistakes this module prevents

If you wrote...	What would happen	What the code does instead
`to_excel(r'C:\...\report.xlsx')`	Two users overwrite each other's file	BytesIO, bytes returned
`return buffer.getvalue() inside the with`	A truncated, unreadable .xlsx	getvalue() after the block
`bands: list = []`	ValueError at import (shared default)	field(default_factory=list)
`ws.cell(1, 2).value = ... inside a band`	Ignored: a merged range shows only its first cell	Title, font and fill on ws.cell(1, c1)
`Header styled on row 1 always`	Banded sheets style the band as a header	header_row from spec.bands
`Rounding money in pandas`	Exact values lost; text columns break	number_format, values unchanged
`money_cols=(5, 6, 7) for activity`	Wrong columns formatted when Name is missing	col_numbers(df, names)
`if summary:`	'truth value of a DataFrame is ambiguous'	if summary is not None
`Building column letters by hand`	Breaks after column Z	cell.column_letter

Part 5. Checkpoints

These tests open the bytes with openpyxl and check real cells. They pass against the project's files.

```
from io import BytesIO

import pandas as pd
from django.test import SimpleTestCase
from openpyxl import load_workbook

from activity.excel import _client_bands
from core.excel import MONEY_FORMAT, SheetSpec, col_numbers, workbook_bytes


class ExcelCheckpoints(SimpleTestCase):
    def setUp(self):
        self.df = pd.DataFrame({"Name": ["x"], "Amount": [12.5]})
        data = workbook_bytes([SheetSpec("Banded", self.df, bands=[("Band", 1, 2, "1F3864")], money_cols=(2,)),
                               SheetSpec("Plain", self.df)])
        self.data = data
        self.wb = load_workbook(BytesIO(data))

    def test_it_is_an_xlsx_and_reads_back(self):
        self.assertTrue(self.data.startswith(b"PK"))
        self.assertEqual(list(pd.read_excel(BytesIO(self.data), sheet_name="Banded", header=1).columns), ["Name", "Amount"])

    def test_band_merged_and_filled(self):
        ws = self.wb["Banded"]
        self.assertIn("A1:B1", [str(r) for r in ws.merged_cells.ranges])
        self.assertEqual(ws["A1"].fill.fgColor.rgb, "001F3864")      # the first cell carries the style
        self.assertEqual(type(ws["B1"]).__name__, "MergedCell")       # the rest are placeholders

    def test_money_format_and_frozen_headers(self):
        self.assertEqual(self.wb["Banded"]["B3"].number_format, MONEY_FORMAT)
        self.assertEqual(self.wb["Banded"].freeze_panes, "B3")
        self.assertEqual(self.wb["Plain"].freeze_panes, "B2")

    def test_col_numbers_skips_missing(self):
        self.assertEqual(col_numbers(pd.DataFrame(columns=["A", "Deposit"]), ["Deposit", "Missing"]), (2,))

    def test_activity_bands_follow_the_columns(self):
        df = pd.DataFrame(columns=["Client ID", "Manager", "Bills", "Trading Days", "Deposit", "Withdraw",
                                   "Net Flow", "Status"])
        self.assertEqual([b[1:3] for b in _client_bands(df)], [(1, 2), (3, 4), (5, 7), (8, 8)])
```

Explain it back

Answer each in one or two sentences, without looking. The answers follow.

1. Why does workbook_bytes return bytes instead of writing a file?
2. Why must buffer.getvalue() come after the with block?
3. What does startrow=1 mean, and which Excel row does the header land on?
4. Why does SheetSpec use field(default_factory=list)?
5. In a merged band, which cell's value and style does Excel show?
6. Why use number_format instead of rounding the numbers?
7. Why does activity find its band edges with get_loc('Bills')?
8. Why is it 'if summary is not None' and not 'if summary'?

Answers

1. So two users never write the same path, and the view can store the bytes wherever the FileField says.
2. ExcelWriter writes and closes the zip file when the block ends; before that the buffer holds an unfinished file.
3. Leave one 0-based row empty; the header lands on Excel row 2, under the band.
4. Each SheetSpec needs its own list; a plain [] default would be shared, so dataclasses forbid it.
5. The first (top-left) cell's. openpyxl turns the others into MergedCell placeholders, so the title, font and fill belong on ws.cell(1, c1).
6. The values stay exact (sums are right), and text cells are unaffected.
7. Optional columns before Bills shift everything; Bills' position gives the number of information columns.
8. A DataFrame has no single True/False value; 'is not None' asks whether a summary exists at all.

Appendix. The files, with line numbers

core/excel.py

```
"""Excel output shared by every report: the MIS look, written in memory."""
from dataclasses import dataclass, field
from io import BytesIO

import pandas as pd
from openpyxl.styles import Alignment, Border, Font, PatternFill, Side

MONEY_FORMAT = '#,##0.00;[Red]-#,##0.00;"-"'


@dataclass
class SheetSpec:
    """One sheet of a workbook and how to style it."""
    name: str
    df: pd.DataFrame
    bands: list = field(default_factory=list)      # [(title, first col, last col, colour), ...]; empty = no band row
    widths: list = field(default_factory=list)     # column widths in order; missing ones get 14
    money_cols: tuple = ()                         # 1-based column numbers
    center_cols: tuple = ()


def style_sheet(ws, spec: SheetSpec):
    """Optional coloured bands on row 1, gold headers, borders, number formats, frozen headers, filters."""
    thin = Side(style="thin", color="D0D5DD")
    header_row = 2 if spec.bands else 1
    for title, c1, c2, colour in spec.bands:
        ws.merge_cells(start_row=1, start_column=c1, end_row=1, end_column=c2)
        for c in range(c1, c2 + 1):
            ws.cell(1, c).fill = PatternFill("solid", fgColor=colour)
        ws.cell(1, c1).value = title
        ws.cell(1, c1).font = Font(name="Arial", bold=True, size=12, color="FFFFFF")
        ws.cell(1, c1).alignment = Alignment(horizontal="center", vertical="center")
        ws.row_dimensions[1].height = 24
    for cell in ws[header_row]:
        cell.font = Font(name="Arial", bold=True, size=10)
        cell.fill = PatternFill("solid", fgColor="FFE699")
        cell.alignment = Alignment(horizontal="center", vertical="center", wrap_text=True)
        cell.border = Border(left=thin, right=thin, top=thin, bottom=Side(style="medium"))
    for row in ws.iter_rows(min_row=header_row + 1, max_row=ws.max_row):
        for cell in row:
            cell.font = Font(name="Arial", size=10)
            cell.border = Border(left=thin, right=thin, top=thin, bottom=thin)
            if cell.column in spec.center_cols:
                cell.alignment = Alignment(horizontal="center")
            if cell.column in spec.money_cols:
                cell.number_format = MONEY_FORMAT
    n_cols = len(spec.df.columns)
    for i in range(1, n_cols + 1):
        width = spec.widths[i - 1] if i <= len(spec.widths) else 14
        ws.column_dimensions[ws.cell(header_row, i).column_letter].width = width
    ws.row_dimensions[header_row].height = 36
    ws.freeze_panes = ws.cell(header_row + 1, 2).coordinate
    ws.auto_filter.ref = f"A{header_row}:{ws.cell(header_row, n_cols).column_letter}{max(ws.max_row, header_row)}"


def workbook_bytes(sheets: list[SheetSpec]) -> bytes:
    """Write every sheet with its style and return the finished .xlsx as bytes."""
    buffer = BytesIO()
    with pd.ExcelWriter(buffer, engine="openpyxl") as writer:
        for spec in sheets:
            spec.df.to_excel(writer, sheet_name=spec.name, index=False, startrow=1 if spec.bands else 0)
            style_sheet(writer.sheets[spec.name], spec)
    return buffer.getvalue()


def col_numbers(df: pd.DataFrame, names) -> tuple:
    """1-based column numbers of the named columns that exist, for money_cols / center_cols."""
    return tuple(df.columns.get_loc(n) + 1 for n in names if n in df.columns)
```

Lines	Block	Purpose
8	`MONEY_FORMAT`	Positive; red negative; dash for zero
11-19	`SheetSpec`	The description of one sheet
22-33	`style_sheet(): bands`	Header row, merged coloured bands
34-46	`style_sheet(): cells`	Headers, borders, formats
47-53	`style_sheet(): finishing`	Widths, frozen headers, filter
56-63	`workbook_bytes()`	All sheets into bytes
66-68	`col_numbers()`	Column numbers by name

commission/excel.py

```
"""The broker commission workbook: which sheets, which bands. The styling itself lives in core.excel."""
from core.excel import SheetSpec, workbook_bytes

BROKER_BANDS = [("Admin Panel Broker File", 1, 4, "1F3864"),
                ("Admin Panel Ledger File", 5, 8, "375623"),
                ("MIS Calculation", 9, 11, "843C0C")]
CLIENT_BANDS = [("Admin Panel Broker File", 1, 4, "1F3864"),
                ("Admin Panel Ledger File", 5, 7, "375623"),
                ("MIS Calculation", 8, 9, "843C0C")]


def build_workbook(brokers, clients) -> bytes:
    return workbook_bytes([
        SheetSpec("Brokers", brokers, bands=BROKER_BANDS,
                  widths=[26, 12, 40, 11, 15, 15, 14, 17, 11, 16, 16],
                  money_cols=(5, 6, 7, 8, 10, 11), center_cols=(4, 9)),
        SheetSpec("Clients", clients, bands=CLIENT_BANDS,
                  widths=[26, 15, 15, 17, 14, 14, 12, 14, 13],
                  money_cols=(5, 6, 7, 8, 9), center_cols=(3,)),
    ])
```

Lines	Block	Purpose
4-9	`Band constants`	Fixed layout
12-20	`build_workbook()`	Two banded sheets

activity/excel.py

```
"""The client activity workbook. Bands follow whichever optional client columns were uploaded."""
from core.excel import SheetSpec, col_numbers, workbook_bytes

MONEY = ["Deposit", "Withdraw", "Net Flow"]


def _client_bands(df):
    n_info = df.columns.get_loc("Bills")                    # Client ID + optional columns
    return [("Client List", 1, n_info, "1F3864"),
            ("Trading", n_info + 1, n_info + 2, "375623"),
            ("Money (month)", n_info + 3, n_info + 5, "375623"),
            ("Result", n_info + 6, n_info + 6, "843C0C")]


def build_workbook(clients, summary=None) -> bytes:
    sheets = [SheetSpec("Clients", clients, bands=_client_bands(clients),
                        widths=[14] + [22] * (clients.columns.get_loc("Bills") - 1) + [10, 12, 14, 14, 14, 13],
                        money_cols=col_numbers(clients, MONEY),
                        center_cols=col_numbers(clients, ["Bills", "Trading Days", "Status"]))]
    if summary is not None:
        sheets.append(SheetSpec("Manager Summary", summary,
                                widths=[18, 18, 13, 10, 13, 14, 10],
                                money_cols=col_numbers(summary, ["Net Flow"]),
                                center_cols=col_numbers(summary, ["Total Clients", "Traded", "Trading Days", "Traded %"])))
    return workbook_bytes(sheets)
```

Lines	Block	Purpose
7-12	`_client_bands()`	Bands from Bills' position
15-25	`build_workbook()`	Clients + optional summary